19: Binary Logistic Regression

Stat 230 - Spring 2026

1 Fitting a Binary Logistic Regression Model

Fitting a binary logistic regression model in R is similar to fitting a linear regression model. To guide you through this process, let's revisit the fake news dataset from class.

To load the data, run the following code chunk:

```
# news <- read.csv("https://aluby.github.io/stat230/data/fake_news_bayesrules.csv")
news <- read.csv(here::here("data/fake_news_bayesrules.csv"))
```

Binary logistic regression requires a binary response variable. In this case, we will use the `type_num` variable, which indicates whether the article was labeled as fake (1 = yes, 0 = no). Be sure to always check that your response variable is coded correctly before modeling. In this case, we want to make sure that `type` and `type_num` match up with our understanding of how the numeric variable is coded.

Next, we will fit a binary logistic regression model using the `glm()` function in R. The syntax is similar to that of the `lm()` function, but we need to specify the `family` argument as `binomial` to indicate that we are performing logistic regression. Here, we will model the probability of an article being fake news by the number of words in the title.

```
news_glm <- glm(type_num ~ title_words, data = news, family = binomial)
```

You can still use the `summary()` function to view the results of the model fit:

```
summary(news_glm)
```

```

Call:
glm(formula = type_num ~ title_words, family = binomial, data = news)
```

```
Coefficients:
             Estimate Std. Error z value Pr(>|z|)
(Intercept)  -2.2058     0.6090  -3.622 0.000292 ***
title_words   0.1588     0.0510   3.114 0.001844 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 201.90  on 149  degrees of freedom
Residual deviance: 191.35  on 148  degrees of freedom
AIC: 195.35

Number of Fisher Scoring iterations: 4
```

Much of the regression output looks the same as in linear regression, but there are some key differences. The main differences are:

- The test statistic is no labeled `z value` instead of `t value`.
- `A Null deviance` and `Residual deviance` are reported instead of `Multiple R-squared` and `Adjusted R-squared`.

2 Making predictions

We can also use R to calculate the predicted probabilities from our fitted logistic regression model using the `predict()` function. Below is an example where we are calculating the predicted probability that a article that has 22 words in the title is fake news

```
predict(news_glm, newdata = data.frame(title_words = 22), type = "response")
```

```
       1
0.783819
```

Notice that we are specifying the data (x values) use for prediction using the same argument as in linear regression: we create a data frame with columns named identically to the actual data set used to fit the mode. We also must specify `type = "response"` to get probabilities. If you omit this argument, you will get a prediction on the log-odds scale, which isn't a probability!

```
predict(news_glm, newdata = data.frame(title_words = 22))
```

```
       1
1.288062
```

3 Your turn

Now, fit a logistic regression model using all three variables we've discussed today (`title_has_excl`, `title_caps_percent`, and `title_words`) and report the `summary()`

```
news_glm_2 <- glm(type_num ~ ___ + ___ + ___, data = news, family = binomial)
summary(___)
```

(1) Write out the fitted model (on the log-odds scale) for articles (A) with an exclamation in the title and (B) without an exclamation in the title

(2) Write out interpretations for all slope coefficients in the model in terms of the **odds** of being fake news

(3) Based on what we know about OLS (**o**rdinary **l**east **s**quares, which includes both SLR and MLR), which predictors appear to be most important for this model?

(4) What questions do you have about the `summary()` output

---

4 Function quick reference

The following table summarizes the functions discussed above:

Function	Purpose
`glm(formula, data, family = binomial)`	Fit a binary logistic regression model
`summary(model)`	View model summary and Wald tests
`predict(model, newdata, type = "response")`	Make predictions on the probability scale